



INTRODUÇÃO AOS DESIGN PATTERNS

Sumário

- Definições
 - Caracterização
 - Padrões GoF
-
- Descrição de Design Patterns
 - Catálogos de Design Patterns
 - Objectos e reutilização
 - Vantagens e riscos
 - Escolha de Design Patterns

O que é um padrão?

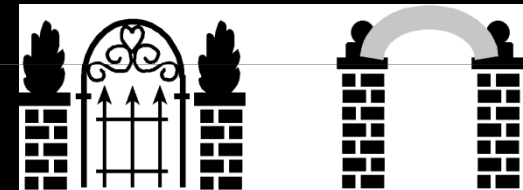
- Definição genérica: referência, modelo
 - Unidades de medida
 - Comportamentos
 - Pedagogias
 - [...]
- Padrões gráficos: repetição de elementos que no seu todo formam eles próprios um desenho identificável
- **Conceito-chave: reutilização** (em ambos os casos acima)

Desenvolvimento de software

- Objectos e classes, hierarquia, interfaces, componentes
- Escolher a granularidade adequada
- Reutilização implica quase sempre adaptação em menor ou maior escala
- A eficácia da reutilização depende muito de quão experiente é o programador

Origem dos Design Patterns

- Origem nos Padrões de Arquitectura de Christopher Alexander (1977)
 - Estruturas para resolver problemas semelhantes (portões, salas de espera) têm elementos semelhantes



- *Gang-of-Four (GoF)*: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*.
 - Transpuseram os conceitos de Alexander para o desenvolvimento de software

Padrões arquiteturais (software)

- Descrevem soluções arquiteturais de problemas de Engenharia de Software
- Alguns exemplos:
 - Layers
 - Model-View-Controller
 - Peer-to-peer
 - Service-Oriented Architecture (SOA)

Design Patterns: o que são?

- Definição geral (comum ao caso da Arquitectura): solução para um problema num contexto
- Soluções genéricas, documentadas (**provadas**) e reutilizáveis, que se podem aplicar em problemas recorrentes
- Exemplos de código (*object-oriented*) para essas soluções

Características de 1 Padrão

- **Nome:** deve identificá-lo inequivocamente e fazer parte de um vocabulário facilmente reconhecível entre colegas
- **Problema:** descreve as situações (problema e contexto) em que é aplicável
- **Solução:** descreve os elementos a usar, bem como os seus relacionamentos, responsabilidades e colaborações
- **Consequências:** resultados esperados, questões de flexibilidade, portabilidade, escalabilidade. Isto pode ter implicações na escolha do padrão.

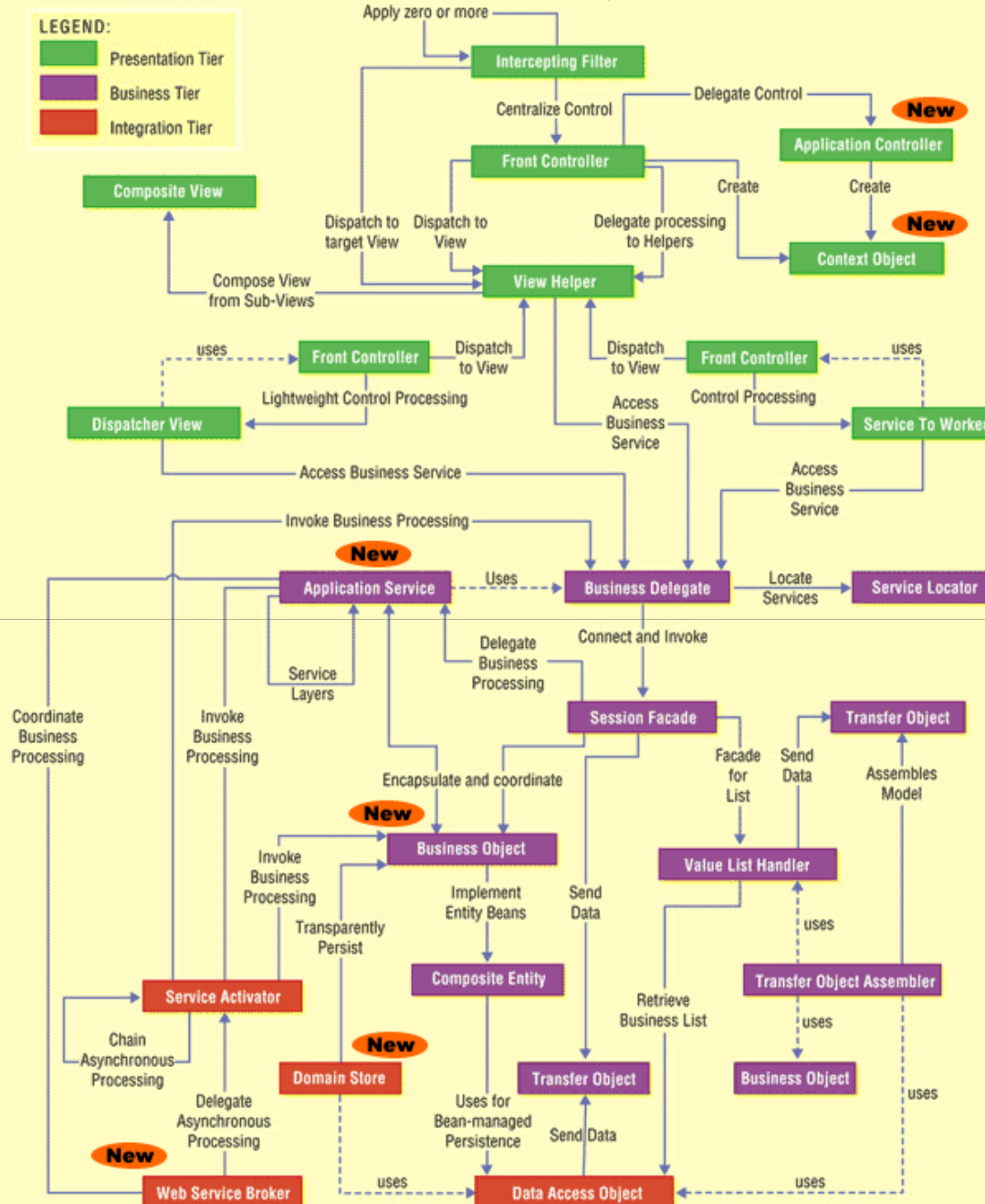
Descrição GoF de Design Patterns

- **Nome**
- **Intenção**: breve descrição do objectivo
- [Nomes alternativos]
- **Motivação**: definição do problema
- **Aplicabilidade**
- **Estrutura**: diagramas de classes e de interacção
- **Participantes**: classes e objectos do padrão e as suas responsabilidades
- **Colaborações**: como interagem os participantes
- **Consequências**
- **Implementação**: dicas e alertas, técnicas, especificidades da linguagem de programação
- **Código de exemplo**: fragmentos ilustrativos de alguns aspectos
- **Utilizações conhecidas**
- **Padrões relacionados**

Catálogos de padrões

- Catálogo GoF
- J2EE - <http://www.corej2eepatterns.com/Patterns2ndEd/index.htm>
- Microsoft .NET - <http://msdn.microsoft.com/en-us/library/ms998469.aspx>
- ServerSide.com - <http://www.theserverside.com/patterns/index.tss>
- POSA (extensão do GoF) - <http://www.vico.org/pages/PatronsDisseny.html>

Core J2EE Patterns, 2nd Edition



Catálogo GoF

- 23 padrões
- Catalogação
 - **Propósito**
 - **Criação** de objectos
 - **Estrutural** – composição de classes ou objectos
 - **Comportamental** – interacção e distribuição de responsabilidades entre as classes ou objectos
 - **Âmbito**
 - **Classe** – relacionamentos estáticos entre classes e sub-classes (mecanismos de herança de classes); fixados em *compile-time*
 - **Objecto** – relacionamentos dinâmicos entre objectos; ocorrem em *run-time*

Classificação de padrões GoF

		Propósito		
		Criação	Estrutural	Comportamental
Âmbito	Classe	Factory Method	Adapter (class)	Interpreter Template Method
	Objecto	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Objectos – conceitos importantes

- Métodos
- Encapsulamento de dados (dados internos acessíveis?)
- Granularidade
- Interfaces
- Implementações
- **Reutilização**
 - Herança de classes – sub-classes estendem classes – *white-box reuse*
 - Composição de objectos, através da obtenção de referências (*run-time*) – *black-box reuse* (só “vê” a interface)
 - Delegação – objecto retorna valor que obteve de um outro objecto
 - Tipos parametrizados / **Templates** – diversos tipos suportados na mesma classe

Reutilização – antecipar mudança!

- Requisitos dos sistemas podem mudar
- Projecto de software deve contemplar este risco
 - Não o fazer pode implicar reimplementações!
- **Design Patterns** ajudam a evitar estes problemas – permitem variações de uns aspectos do sistema de forma independente dos outros → **Robustez!**

Erros comuns (padrões úteis)

- Criação de 1 objecto especificando explicitamente a sua classe → criação indirecta/interfaces (Abstract Factory, Factory Method, Prototype)
 - Exemplo da GUI e do “look and feel”
- Dependência de operações específicas – problema semelhante ao anterior (Chain of Responsibility, Command)
 - Exemplo da ajuda contextual
- Dependência de plataformas de hardware e software (Abstract Factory, Bridge)

Erros comuns (padrões úteis)

Continuação

- Dependência de representações e implementações de objectos (Abstract Factory, Bridge, Memento, Proxy)
 - Por exemplo, ter que saber localização exacta de um objecto remoto
- Dependências algorítmicas (Builder, Iterator, Strategy, Template Method, Visitor)
- Acoplamento forte de classes – difícil reutilização quando isoladas → sistemas monolíticos (Abstract Factory, Bridge, Chain of Responsibility, Command, Facade, Mediator, Observer)

Erros comuns (padrões úteis)

Continuação

- Estender funcionalidades por herança – pode conduzir a excesso de classes e sub-classes, muitos “overrides” e esquemas complexos de subclasses → usar mais a composição e a delegação (Bridge, Chain of Responsibility, Composite, Decorator, Observer, Strategy)
- Incapacidade para alterar classes – software comercial, muitas subclasses (Adapter, Decorator, Visitor)

Reutilização de classes

- **Toolkits** – bibliotecas de classes reutilizáveis, com funcionalidades genéricas
 - Exemplos: listas, stacks
- **Frameworks** – bibliotecas de classes reutilizáveis, com funcionalidades úteis para uma determinada categoria de aplicações
 - Exemplos: Swing, JMF, JDBC
- Design Patterns? → reutilização de soluções!

Design Patterns - discussão

Vantagens

- Reutilização de soluções provadas
- Robustez
- Terminologia comum
- Abstracção de alto nível
- Boa utilização dos princípios básicos de objectos
- Design para a mudança

Riscos

- Tentação a usar padrões onde não se adequam
- Limitação de criatividade
- Eventual exagero na sua utilização pode conduzir a complexidade desnecessária

Como escolher os padrões?

- Lembrar que os Design Patterns resolvem problemas de design
- Olhar para o campo da Intenção na descrição do padrão
- Atender ao propósito
- Estudar relacionamentos entre padrões
- Verificar possíveis causas de reimplementação
- Antecipar o que poderá ser feito para evitar reimplementação

1500

